# Informationstechnologie I - Skript zu Termin 3: Funktionen und eigene Methoden

## 1. Einführung: Wiederkehrenden Aufgaben einen Namen geben

Bisher wurden Anweisungen direkt hintereinander geschrieben: Variablen anlegen, Werte berechnen, Bedingungen prüfen und Ergebnisse ausgeben. Wenn eine Berechnung an mehreren Stellen benötigt wird, lässt sie sich zunächst einfach wiederholen:

```
int firstWidth = 4;
int firstHeight = 3;
int firstArea = firstWidth * firstHeight;
Console.WriteLine(firstArea);

int secondWidth = 5;
int secondHeight = 2;
int secondArea = secondWidth * secondHeight;
Console.WriteLine(secondArea);
```

Die Ausgabe lautet `12` und danach `10`. Beide Rechtecke werden nach derselben Regel berechnet: **Breite mal Höhe**. Nur die eingesetzten Maße unterscheiden sich. Breite und Höhe müssen jeweils in derselben Längeneinheit vorliegen; das Ergebnis ist dann eine Fläche in der entsprechenden Flächeneinheit.

Das Kopieren der Formel funktioniert hier. Bei umfangreicheren Berechnungen wird jedoch schwerer erkennbar, welche Codeabschnitte dieselbe Aufgabe erledigen. Muss die Berechnung später geändert werden, müssen alle Kopien gefunden und angepasst werden.

Eine **Methode** fasst Anweisungen unter einem Namen zusammen. Der Name beschreibt die Aufgabe, die Anweisungen beschreiben ihre Umsetzung. Über einen **Aufruf** kann das Programm diese Aufgabe ausführen lassen. Eine Methode lässt sich mit einer Arbeitsanleitung vergleichen: Die Anleitung legt fest, was zu tun ist. Erst der Auftrag, sie jetzt auszuführen, setzt die beschriebenen Schritte in Gang.

Mit `Console.WriteLine(...)` wurde bereits eine vorhandene Methode verwendet. In diesem Termin werden eigene Bausteine geschrieben. Zunächst geht es um den Ablauf eines Aufrufs, anschließend um Eingabewerte, Ergebnisse und verschiedene Übergabearten. Zum Schluss wird betrachtet, wie eine Methode sich selbst aufrufen kann.

Die Beispiele verwenden wie bisher **Top-Level-Statements**: Die direkt auszuführenden Anweisungen stehen in der vorbereiteten C#-Datei. Diese Anweisungen werden im Skript als **Hauptprogramm** bezeichnet. Die eigenen Bausteine in dieser Form heißen in C# genauer **lokale Funktionen**. An ihnen werden die Grundlagen von Aufruf, Parametern und Rückgabe erklärt.

Verwenden Sie die vollständigen Beispiele jeweils einzeln als Inhalt der vorbereiteten Datei. Sie bilden kein fortlaufend zu ergänzendes Programm. Werden mehrere Beispiele untereinander eingefügt, können unter anderem doppelte Variablen- und Funktionsnamen entstehen. Der bekannte Arbeitsablauf bleibt hilfreich: **Ausgabe vorhersagen → Code ausführen → Ergebnis erklären**.

## 2. Definition und Aufruf unterscheiden

Ein Programm ohne eigenen Baustein könnte so aussehen:

```
Console.WriteLine("Start");
Console.WriteLine("Hallo!");
Console.WriteLine("Ende");
```

Alle drei Anweisungen werden der Reihe nach ausgeführt. Nun wird die Begrüßung in einen eigenen Baustein mit dem Namen `SayHello` verschoben:

```
Console.WriteLine("Start");
SayHello();
Console.WriteLine("Ende");

void SayHello()
{
    Console.WriteLine("Hallo!");
}
```

Die Ausgabe ist weiterhin:

```
Start
Hallo!
Ende
```

Der Quellcode enthält nun zwei unterschiedliche Bestandteile. Die **Definition** von `SayHello` legt fest, welche Anweisungen zu diesem Baustein gehören. Der **Aufruf** `SayHello();` führt diese Anweisungen aus. Eine Definition allein erzeugt keine Ausgabe.

### Warum darf die Definition unter dem Aufruf stehen?

Der **Compiler** prüft und übersetzt den Quellcode vor der Ausführung. Dabei kann er den Namen `SayHello` der zugehörigen Definition zuordnen, auch wenn diese weiter unten steht. Die Reihenfolge der Definitionen bestimmt daher nicht, wann ihre Anweisungen ausgeführt werden. Diese Platzierung lokaler Funktionen ist auch in der C#-Dokumentation zu lokalen Funktionen beschrieben.

Als Denkmodell kann man sich ein Verzeichnis vorstellen: Zum Namen `SayHello` gehört die Anleitung „Gib `Hallo!` aus“. Das laufende Programm muss dafür nicht erst alle Zeilen durchgehen und die Anleitungen einsammeln. Die Zuordnung wird bereits beim Übersetzen geprüft. Für Variablen gelten eigene Regeln; die Möglichkeit, eine Funktionsdefinition unter ihren Aufruf zu schreiben, lässt sich nicht einfach auf Variablendeklarationen übertragen.

### Einen Aufruf Schritt für Schritt nachvollziehen

| Schritt | Was wird ausgeführt? | Wirkung |
|---|---|---|
| 1 | `Console.WriteLine("Start");` | `Start` wird ausgegeben. |
| 2 | `SayHello();` | Die Ausführung von `SayHello` beginnt. Die Fortsetzung im Hauptprogramm wird gemerkt. |
| 3 | `Console.WriteLine("Hallo!");` in `SayHello` | `Hallo!` wird ausgegeben. |
| 4 | Das Ende von `SayHello` wird erreicht. | Der Aufruf ist beendet; es geht zum Hauptprogramm zurück. |
| 5 | `Console.WriteLine("Ende");` | `Ende` wird ausgegeben. |

Während `SayHello` ausgeführt wird, wartet das Hauptprogramm an dieser Stelle. Nach dem Aufruf geht es mit seiner nächsten Anweisung weiter. Das Hauptprogramm beginnt also nicht erneut von vorn. Ebenso wird nach seiner letzten Ausgabe die darunterstehende Definition nicht noch einmal als normaler Programmablauf durchlaufen.

„Von oben nach unten“ gilt weiterhin für die Anweisungen im jeweiligen Ablauf. Ein Methodenaufruf führt zusätzlich die Anweisungen des benannten Bausteins aus und kehrt anschließend zur Aufrufstelle zurück.

## 3. Aufbau und mehrfache Verwendung einer Methode

Der bereits verwendete Baustein lässt sich nun in seine Bestandteile zerlegen:

```
SayHello();

void SayHello()
{
    Console.WriteLine("Hallo!");
}
```

| Bestandteil | Bedeutung |
|---|---|
| `void` | Die Methode liefert keinen Rückgabewert. |
| `SayHello` | Der Name des Bausteins. |
| `()` in der Definition | Diese Methode benötigt keine Eingabewerte. |
| `{ ... }` | Der Methodenkörper mit den auszuführenden Anweisungen. |
| `SayHello();` | Der Aufruf im Hauptprogramm. |

Der erste Teil der Definition, hier `void SayHello()`, wird als **Methodenkopf** bezeichnet. Danach folgt der Methodenkörper zwischen geschweiften Klammern. Hinter dem Methodenkopf steht kein Semikolon. Der Aufruf und die Ausgabeanweisung im Körper werden dagegen jeweils mit `;` beendet.

**`void` bedeutet ausschließlich, dass kein Wert zurückgegeben wird.** Eine solche Methode darf trotzdem etwas auf die Konsole schreiben. Sie kehrt am Ende auch ohne ausgeschriebenes `return` zum

Aufrufer zurück.

Methodennamen werden üblicherweise in **PascalCase** geschrieben: Jedes Wort beginnt mit einem Großbuchstaben, beispielsweise `SayHello` oder `PrintCourseInfo`. Variablen und Parameter verwenden wie bisher **camelCase**, etwa `firstWidth`. C# unterscheidet Groß- und Kleinschreibung; `SayHello` und `sayHello` sind verschiedene Namen.

## Eine Definition, mehrere Aufrufe

```
SayHello();
Console.WriteLine("Pause");
SayHello();

void SayHello()
{
    Console.WriteLine("Hallo!");
}
```

Die Ausgabe lautet:

```
Hallo!
Pause
Hallo!
```

Jeder Aufruf beginnt erneut am Anfang des Methodenkörpers. Nach dem ersten Aufruf geht es zur Ausgabe `Pause` zurück, nach dem zweiten ist das Hauptprogramm hier fertig. Dafür wird nur eine Definition benötigt.

## Die Aufrufe bestimmen die Reihenfolge

```
PrintFooter();
PrintHeader();

void PrintHeader()
{
    Console.WriteLine("Willkommen!");
}

void PrintFooter()
{
    Console.WriteLine("Bis bald!");
}
```

Obwohl `PrintHeader` zuerst definiert ist, wird zunächst `Bis bald!` und danach `Willkommen!` ausgegeben. Entscheidend sind die Aufrufe im Hauptprogramm. Auch die Namen `Header` und `Footer`

besitzen für C# keine besondere Bedeutung. Wird ein Baustein gar nicht aufgerufen, führt das Programm seinen Körper nicht aus.

## 4. Parameter: Einer Methode Werte mitgeben

Bisher wird bei jedem Aufruf derselbe feste Text ausgegeben. Ein **Parameter** ermöglicht es, dieselbe Anleitung mit unterschiedlichen Werten zu verwenden:

```
Greet("Mia");
Greet("Noah");

void Greet(string name)
{
    Console.WriteLine("Hallo, " + name + "!");
}
```

Die Ausgabe lautet:

```
Hallo, Mia!
Hallo, Noah!
```

In der Definition steht nun `string name` zwischen den runden Klammern. Damit wird ein Parameter mit dem Typ `string` und dem Namen `name` festgelegt. Beim ersten Aufruf erhält er den Text `"Mia"`, beim zweiten den Text `"Noah"`. Der Parameter gehört zum jeweiligen Aufruf.

Das Zeichen `+` setzt hier Texte zusammen. Aus `"Hallo, " + "Mia" + "!"` entsteht `"Hallo, Mia!"`. Das Leerzeichen nach dem Komma ist Teil des ersten Textes. Diese Verwendung von `+` heißt **Textverkettung**; bei Zahlen wurde dasselbe Zeichen bereits für die Addition verwendet.

Ein Eingabewert für eine Methode ist nicht automatisch eine Tastatureingabe. Hier stehen die Texte direkt im Code. Die Methode erhält die Werte durch ihren Aufruf und verwendet selbst kein `Console.ReadLine()`.

### Parameter und Argument

Die Begriffe bezeichnen zwei Seiten derselben Übergabe:

```
string student = "Mia";
Greet(student);

void Greet(string name)
{
    Console.WriteLine("Hallo, " + name + "!");
}
```

- Der **Parameter** steht in der Definition: `string name` beschreibt den erwarteten Wert.

- Das **Argument** steht beim Aufruf: Hier ist das der Ausdruck `student`, dessen aktueller Wert `"Mia"` übergeben wird.

Die Namen `student` und `name` dürfen verschieden sein. Die Zuordnung erfolgt durch den Aufruf und nicht durch übereinstimmende Namen. Ein Argument kann ein Literal, eine Variable oder ein berechneter Ausdruck sein.

### Mehrere Parameter und ihre Reihenfolge

```
PrintDifference(10, 3);
PrintDifference(3, 10);

void PrintDifference(int left, int right)
{
    Console.WriteLine(left - right);
}
```

Das erste Argument wird an `left`, das zweite an `right` übergeben:

| Aufruf | `left` | `right` | Ausgabe |
|---|---|---|---|
| `PrintDifference(10, 3);` | `10` | `3` | `7` |
| `PrintDifference(3, 10);` | `3` | `10` | `-7` |

In der Definition werden die Parameter jeweils mit Typ und Name angegeben. Beim Aufruf stehen nur die Argumente, getrennt durch Kommas. Ihre Anzahl und Typen müssen zur Definition passen. `PrintDifference("10", 3);` wäre beispielsweise ungültig, weil das erste Argument ein Text ist, die Methode dort aber einen `int`-Wert erwartet.

## 5. Rückgabewerte: Ein Ergebnis weiterverwenden

Eine Methode kann ein Ergebnis berechnen und an ihre Aufrufstelle zurückgeben. Damit lässt sich das Ergebnis im Hauptprogramm speichern und weiterverwenden:

```
int result = Add(5, 3);
Console.WriteLine(result);

int Add(int left, int right)
{
    return left + right;
}
```

Vor dem Namen `Add` steht nun `int` statt `void`. Dieser **Rückgabetyp** legt fest, dass die Methode eine ganze Zahl liefert. Die Anweisung `return` berechnet den nachfolgenden Ausdruck, beendet den aktuellen Aufruf und liefert den Wert an die Aufrufstelle zurück.

Der Datenfluss lässt sich so darstellen:

```
Argumente            Methode                    Rückgabewert
5 und 3      →       Add: 5 + 3 berechnen   →     8
```

### Erst den Aufruf auswerten, dann zuweisen

Die Zuweisung `int result = Add(5, 3);` folgt dem bekannten Prinzip: Zuerst wird die rechte Seite ausgewertet, danach das Ergebnis links gespeichert.

1. `Add(5, 3)` wird aufgerufen. `left` erhält `5`, `right` erhält `3`.
2. Die Methode berechnet `left + right`, also `8`.
3. `return` liefert `8` an die Aufrufstelle zurück.
4. Die Zuweisung speichert `8` in `result`.
5. Erst die nächste Anweisung gibt `result` auf der Konsole aus.

Nach der Rückgabe kann man sich die erste Anweisung gedanklich als `int result = 8;` vorstellen. Die Methode `Add` selbst erzeugt keine Konsolenausgabe. Der Name `result` ist ebenfalls keine besondere C#-Vorgabe; er wurde hier als verständlicher Variablenname gewählt.

### Ausgeben und zurückgeben unterscheiden

```
PrintSum(5, 3);
int total = Add(5, 3);

void PrintSum(int left, int right)
{
    Console.WriteLine(left + right);
}

int Add(int left, int right)
{
    return left + right;
}
```

Dieses Programm gibt genau einmal `8` aus: durch `PrintSum`. Anschließend speichert es einen weiteren Wert `8` in `total`, ohne ihn auszugeben.

| Vorgang | Wohin gelangt der Wert? | Wozu dient er? |
|---|---|---|
| `Console.WriteLine(...)` | Als Ausgabe auf die Konsole. | Ein Mensch kann den Wert sehen. |
| `return ...;` | Als Ergebnis an die Aufrufstelle. | Der Programmcode kann den Wert weiterverwenden. |

`int total = PrintSum(5, 3);` wäre ungültig: Die Methode hat den Rückgabetyp `void` und liefert daher keinen Wert für die Zuweisung. Umgekehrt ist `Add(5, 3);` als einzelner Aufruf erlaubt. Sein

Ergebnis wird dann jedoch nicht verwendet und erscheint auch nicht automatisch auf der Konsole.

### Mit einem Rückgabewert weiterrechnen

```
int result = Add(5, 3);
int doubled = result * 2;
Console.WriteLine(doubled);

Console.WriteLine(Add(10, 20));

int Add(int left, int right)
{
    return left + right;
}
```

Die Ausgabe lautet `16` und danach `30`. Im ersten Teil wird der Rückgabewert gespeichert und anschließend verdoppelt. Bei `Console.WriteLine(Add(10, 20));` steht ein Aufruf innerhalb eines anderen Aufrufs. Zuerst wird `Add(10, 20)` ausgewertet. Danach erhält `Console.WriteLine` dessen Ergebnis `30` als Argument.

Ein Rückgabewert kann damit an einer Stelle stehen, an der ein Wert des passenden Typs benötigt wird. Auch `int doubled = Add(5, 3) * 2;` wäre möglich. Separate Zwischenvariablen können allerdings helfen, den Ablauf zu erklären und Fehler zu finden.

## 6. `return` und Bedingungen kombinieren

Ein Rückgabewert muss keine Zahl sein. Eine Prüfmethode kann beispielsweise einen Wahrheitswert liefern:

```
Console.WriteLine(IsAdult(20));

bool IsAdult(int age)
{
    if (age >= 18)
    {
        return true;
    }
    return false;
}
```

Für `20` trifft die Bedingung zu. Die Methode führt `return true;` aus und ist damit für diesen Aufruf beendet. Die spätere Anweisung `return false;` wird in diesem Fall nicht mehr erreicht. `Console.WriteLine` gibt `True` aus.

Für einen Aufruf mit `17` wird der `if`-Block übersprungen. Anschließend liefert `return false;` den Wert `false`; die Ausgabe wäre `False`. Die Schlüsselwörter im Code werden kleingeschrieben, die Konsolendarstellung der Wahrheitswerte beginnt mit einem Großbuchstaben.

Ein `else` ist hier nicht nötig, weil das erste `return` den aktuellen Aufruf bereits beendet. **Eine Methode mit einem Rückgabetyp muss auf jedem möglichen Weg, auf dem sie normal endet, einen passenden Wert zurückgeben.** Würde das zweite `return` fehlen, gäbe es für `age < 18` keinen Rückgabewert. Der Compiler meldet dann einen Fehler.

### Eine Bedingung liefert bereits einen Wahrheitswert

Der Vergleich `age >= 18` hat selbst den Typ `bool`. Deshalb kann dieselbe Prüfung kürzer geschrieben werden:

```
int age = 20;
if (IsAdult(age))
{
    Console.WriteLine("Zutritt ab 18 erlaubt.");
}

bool IsAdult(int age)
{
    return age >= 18;
}
```

Der Methodenaufruf liefert genau den Wahrheitswert, den `if` benötigt. Ein zusätzlicher Vergleich mit `true` ist nicht erforderlich. Die ausgeschriebene Version und die kurze Version geben für dieselben Alterswerte dieselben Ergebnisse zurück.

Die Methode prüft ausschließlich die Altersgrenze. Ob ein Zutritt tatsächlich erlaubt ist, könnte zusätzlich von anderen Voraussetzungen abhängen. Solche fachlichen Regeln sollten durch den Namen und den verwendeten Text erkennbar bleiben.

## 7. Dieselbe Berechnung mit verschiedenen Werten verwenden

Das Einstiegsbeispiel mit zwei Rechtecken lässt sich nun mit Parametern und Rückgabewert schreiben:

```
int firstArea = CalculateRectangleArea(4, 3);
Console.WriteLine(firstArea);
int secondArea = CalculateRectangleArea(5, 2);
Console.WriteLine(secondArea);

int CalculateRectangleArea(int width, int height)
{
    return width * height;
}
```

Die Ausgabe bleibt `12` und danach `10`. Beim ersten Aufruf erhält `width` den Wert `4` und `height` den Wert `3`. Beim zweiten Aufruf sind es `5` und `2`. Jeder Aufruf berechnet sein eigenes Ergebnis; die Formel steht nur einmal im Code.

Für dieses Beispiel werden ganze Maße ab null vorausgesetzt. Die Methode prüft diese Voraussetzung nicht selbst. Ein Aufruf mit negativen Werten könnte zwar zum Typ `int` passen, würde aber nicht zum vereinbarten Anwendungsfall passen. **Ein passender Datentyp allein garantiert noch keinen fachlich sinnvollen Wert.**

## 8. Gültigkeitsbereich: Wo sind Variablen sichtbar?

Parameter und Variablen innerhalb einer Methode gehören zu deren Ausführung:

```
int firstNumber = 5;
int result = DoubleValue(firstNumber);
Console.WriteLine(result);

int DoubleValue(int value)
{
    int doubled = value * 2;
    return doubled;
}
```

Die Ausgabe lautet `10`. Der Parameter `value` und die lokale Variable `doubled` können nur innerhalb von `DoubleValue` verwendet werden. Eine zusätzliche Anweisung `Console.WriteLine(doubled);` im Hauptprogramm wäre ungültig, weil der Name dort nicht sichtbar ist.

Dieser Bereich der Sichtbarkeit heißt **Gültigkeitsbereich** oder **Scope**. Er beschreibt, an welchen Stellen im Quellcode ein Name verwendet werden darf. Das Hauptprogramm erhält mit `return` den berechneten Wert, aber keinen Zugriff auf die lokale Variable `doubled`. Die eigene Variable `result` speichert diesen Wert anschließend unter einem anderen Namen.

Jeder Aufruf besitzt seine eigenen Parameter und lokalen Variablen. Bei einem weiteren Aufruf von `DoubleValue` wird die Berechnung erneut mit den dann übergebenen Werten ausgeführt. Ein vorheriges Ergebnis wird dadurch nicht automatisch aktualisiert.

Lokale Funktionen können grundsätzlich auch auf bestimmte Variablen des umgebenden Codes zugreifen. In den Beispielen werden benötigte Werte bewusst über Parameter übergeben. So lässt sich direkt am Aufruf erkennen, welche Eingaben die Berechnung verwendet.

## 9. Wertübergabe: Mit einer Kopie arbeiten

Standardmäßig erhält ein Parameter eine **Kopie des Argumentwertes**. Bei einem `int` sind die äußere Variable und der Parameter damit zwei getrennte Variablen mit zunächst gleichem Zahlenwert:

```
int number = 5;
PrintIncreased(number);
Console.WriteLine(number);

void PrintIncreased(int value)
{
    value = value + 1;
```

```
    Console.WriteLine(value);
}
```

Die Ausgabe lautet:

```
6
5
```

| Zeitpunkt | `number` im Hauptprogramm | `value` im Methodenaufruf |
|---|---|---|
| Vor dem Aufruf | 5 | Noch kein Aufruf begonnen. |
| Beim Beginn des Aufrufs | 5 | 5 |
| Nach `value = value + 1;` | 5 | 6 |
| Nach der Rückkehr | 5 | Der Aufruf ist beendet. |

Die Zuweisung an `value` verändert nur den Parameter. Die äußere Variable `number` bleibt bei `5`. Das ist dasselbe Kopierprinzip, das bereits von Zuweisungen zwischen zwei `int`-Variablen bekannt ist.

Die Aussage bezieht sich hier auf den Zahlenwert und die Zuweisung an den Parameter. Daraus sollte nicht die allgemeine Regel abgeleitet werden, dass ein Methodenaufruf außerhalb seines Körpers grundsätzlich nichts verändern kann. Für die folgenden Beispiele genügt die Unterscheidung zwischen der Kopie eines `int`-Wertes und dem direkten Zugriff auf eine Variable mit `ref`.

### Einen neuen Wert zurückgeben und außen zuweisen

Soll `number` nach der Berechnung einen neuen Wert enthalten, kann das Hauptprogramm das Ergebnis ausdrücklich zuweisen:

```
int number = 5;
number = Increase(number);
Console.WriteLine(number);

int Increase(int value)
{
    return value + 1;
}
```

Hier wird `6` ausgegeben. Zunächst liest der Aufruf rechts den bisherigen Wert `5`. Die Methode liefert `6`. Erst danach ersetzt die Zuweisung links den alten Wert von `number` durch dieses Ergebnis.

Die Methode verändert `number` also nicht selbst. Würde lediglich `Increase(number);` ohne anschließende Zuweisung ausgeführt, bliebe `number` bei `5`. Ein berechneter und zurückgegebener Wert muss im aufrufenden Code verwendet werden, damit er dort eine Wirkung hat.

## 10. Kleine Methoden miteinander verbinden

Mehrere Methoden können nacheinander verschiedene Teile einer Verarbeitung übernehmen:

```
int age = 20;
bool adult = IsAdult(age);
string text = GetAgeMessage(adult);
Console.WriteLine(text);

bool IsAdult(int age)
{
    return age >= 18;
}

string GetAgeMessage(bool adult)
{
    if (adult)
    {
        return "Volljährig";
    }
    return "Minderjährig";
}
```

Die Verarbeitung erfolgt in drei Schritten. `IsAdult` prüft das Alter und liefert `true`. `GetAgeMessage` erhält diesen Wahrheitswert und bestimmt daraus den Text `"Volljährig"`. Erst das Hauptprogramm gibt den Text aus.

```
20       →       true       →       "Volljährig"       →
Konsolenausgabe
         IsAdult            GetAgeMessage              WriteLine
```

Jeder Baustein hat eine klar beschriebene Aufgabe: prüfen, einen Text bestimmen oder ausgeben. Dadurch lässt sich der Datenfluss im Hauptprogramm lesen, ohne sofort alle Einzelheiten der Methoden betrachten zu müssen. Die Rückgabetypen zeigen zugleich, welche Art von Wert der nächste Schritt erhält.

## 11. Refactoring und sinnvolle Methodengestaltung

Eine bereits funktionierende Berechnung kann nachträglich in eine Methode ausgelagert werden. Vorher könnte der Code so aussehen:

```
int width = 4;
int height = 3;
int area = width * height;
Console.WriteLine(area);
```

Die Ausgabe ist `12`. Nach dem Auslagern lautet das Programm:

```
int width = 4;
int height = 3;
int area = CalculateRectangleArea(width, height);
Console.WriteLine(area);

int CalculateRectangleArea(int width, int height)
{
    return width * height;
}
```

Die Ausgabe bleibt `12`. Eine solche Änderung des Codeaufbaus bei gleichbleibendem Verhalten heißt **Refactoring**. Die Berechnung erhält einen Namen und kann nun an mehreren Stellen aufgerufen werden.

Beim Auslagern hilft es, zuerst die Aufgabe in einem Satz zu beschreiben: „Berechne die Rechteckfläche aus Breite und Höhe." Daraus ergeben sich die Eingaben `width` und `height` sowie das Ergebnis `area`. Die Eingaben werden zu Parametern, das Ergebnis wird zurückgegeben. Die Ausgabe bleibt im Hauptprogramm.

Zur Kontrolle eignen sich unabhängig bestimmte Ergebnisse:

| Breite | Höhe | Erwartete Fläche |
|---|---|---|
| `4` | `3` | `12` |
| `5` | `2` | `10` |
| `0` | `3` | `0` |

Diese Fälle sollten vor und nach der Änderung dasselbe Ergebnis liefern. Gerade ein Grenzwert wie null ergänzt die üblichen positiven Beispiele sinnvoll.

### Was macht eine Methode verständlich?

Eine Methode sollte eine klar beschriebene Aufgabe erledigen. Ein aussagekräftiger Name erklärt ihre Absicht; die Parameter zeigen die benötigten Werte. Der Rückgabewert stellt das Ergebnis für weitere Verarbeitung bereit.

Berechnung und Konsolenausgabe zu trennen hilft beim Prüfen: Für eine Berechnung lassen sich Eingaben und erwartetes Ergebnis direkt gegenüberstellen. Außerdem kann dasselbe Ergebnis später auf unterschiedliche Weise dargestellt werden.

Nicht jede einzelne Anweisung benötigt eine eigene Methode. Sinnvoll ist ein Baustein vor allem dann, wenn er eine verständliche Teilaufgabe benennt oder mehrfach benötigte Logik an einer Stelle bündelt. Eine Methode macht ein Programm auch nicht automatisch kürzer oder schneller. Der Vorteil liegt hier in einem verständlicheren Aufbau und darin, eine gemeinsame Berechnung zentral ändern zu können.

## 12. Typische Fehler bei Methoden erkennen

Viele Fehler lassen sich durch die Unterscheidung von Definition, Aufruf, Übergabe und Rückgabe eingrenzen:

| Beobachtung | Erste Prüfung |
|---|---|
| Es erscheint keine Ausgabe. | Wurde die Methode aufgerufen? Wird ihr Ergebnis auch ausgegeben? |
| Der Compiler kennt den Namen nicht. | Stimmen Schreibweise, Groß- und Kleinschreibung sowie Gültigkeitsbereich? |
| Der Aufruf wird abgelehnt. | Passen Anzahl, Reihenfolge und Typen der Argumente zur Definition? |
| Ein Ergebnis lässt sich nicht speichern. | Hat die verwendete Methode den Rückgabetyp `void`? |
| Der Compiler fordert einen Rückgabewert. | Liefert die Methode auf jedem normal endenden Weg einen passenden Wert? |
| Eine Variable ist außerhalb der Methode nicht verfügbar. | Wurde eine lokale Variable außerhalb ihres Gültigkeitsbereichs verwendet? |
| Eine äußere Zahl bleibt unverändert. | Wurde nur die Parameterkopie verändert oder ein Rückgabewert ohne Zuweisung verworfen? |

Beginnen Sie mit der ersten Fehlermeldung und der dort angegebenen Stelle. Bei unerwarteten Ergebnissen hilft eine Ablaufspur: Welche Werte werden übergeben? Welche Anweisung läuft als Nächstes? Welcher Wert kommt zurück, und wo wird er gespeichert?

Das folgende Beispiel zeigt zwei bewusst getrennte Werte:

```
int number = 3;
int result = DoubleValue(number);
Console.WriteLine(number);
Console.WriteLine(result);

int DoubleValue(int value)
{
    return value * 2;
}
```

Die Ausgabe lautet `3` und danach `6`. Die Methode berechnet einen neuen Wert; `number` bleibt unverändert. Würde die zweite Zeile `int result = DoubleValue(number) + 1;` lauten, wäre die zweite Ausgabe `7`: Zuerst liefert die Methode `6`, danach wird `1` addiert und das Ergebnis gespeichert.

## 13. `ref`: Eine Variable des Aufrufers verändern

Bisher hat die Methode eine Kopie des übergebenen Zahlenwertes erhalten. Eine Veränderung der äußeren Variable wurde anschließend durch eine Zuweisung im Hauptprogramm vorgenommen. Mit **`ref`** kann eine Methode stattdessen direkt auf die übergebene Variable zugreifen:

```
int number = 5;
IncreaseInPlace(ref number);
Console.WriteLine(number);

void IncreaseInPlace(ref int value)
{
    value = value + 1;
}
```

Die Ausgabe lautet `6`. Der Parameter `value` bezeichnet hier dieselbe Variable wie `number` im Hauptprogramm. Eine Zuweisung an `value` verändert deshalb unmittelbar den Wert dieser Variable. Es wird kein zusätzliches Ergebnis über `return` zurückgegeben; der Rückgabetyp bleibt `void`.

`ref` steht sowohl vor dem Parametertyp in der Definition als auch vor dem Argument beim Aufruf. Die Variable muss vor dem Aufruf bereits einen Wert besitzen. Ein fester Zahlenwert wie `5` kann nicht als `ref`-Argument eingesetzt werden, weil die Methode eine Variable benötigt, auf die sie zugreifen kann.

Am Aufruf `IncreaseInPlace(ref number);` ist damit bereits sichtbar, dass die Methode `number` verändern kann. Sie ist dazu allerdings nicht verpflichtet: Anders als bei `out` muss ein `ref`-Parameter nicht zwingend einen neuen Wert erhalten.

### Zwei Werte tauschen

Ein anschaulicher Einsatz von `ref` ist das Tauschen zweier Variablenwerte:

```
int first = 5;
int second = 10;
Swap(ref first, ref second);
Console.WriteLine(first);
Console.WriteLine(second);

void Swap(ref int left, ref int right)
{
    int temporary = left;
    left = right;
    right = temporary;
}
```

Die Ausgabe lautet `10` und danach `5`. Während dieses Aufrufs bezeichnet `left` die Variable `first` und `right` die Variable `second`.

| Schritt im Methodenkörper | `first` / `left` danach | `second` / `right` danach | `temporary` danach |
|---|---|---|---|
| `int temporary = left;` | `5` | `10` | `5` |
| `left = right;` | `10` | `10` | `5` |

| Schritt im Methodenkörper | `first` / `left` danach | `second` / `right` danach | `temporary` danach |
|---|---|---|---|
| `right = temporary;` | `10` | `5` | `5` |

Die temporäre Variable bewahrt den ursprünglichen linken Wert auf. Ohne diesen Zwischenspeicher wäre nach `left = right;` der bisherige Wert `5` verloren. Eine anschließende Anweisung `right = left;` würde dann ebenfalls `10` zuweisen und keinen Tausch bewirken.

Würden die Parameter und Aufrufe ohne `ref` geschrieben, würden lediglich die beiden lokalen Parameterkopien getauscht. Die Variablen im Hauptprogramm blieben unverändert.

## 14. `out`: Ergebnisse in Variablen schreiben

Mit **`out`** kann eine Methode einen Wert in eine Variable des Aufrufers schreiben, ohne dass diese vorher einen Wert besitzen muss:

```
GetDouble(5, out int result);
Console.WriteLine(result);

void GetDouble(int value, out int doubled)
{
    doubled = value * 2;
}
```

Die Ausgabe lautet `10`. `value` ist ein normaler Eingabeparameter und erhält den Wert `5`. `doubled` ist ein Ausgabeparameter, über den die Methode das Ergebnis in `result` schreibt.

Auch `out` steht in der Definition und beim Aufruf. Die Schreibweise `out int result` deklariert die Ergebnisvariable direkt an der Aufrufstelle. Danach kann das Hauptprogramm `result` verwenden. Alternativ könnte die Variable vorher mit `int result;` deklariert und anschließend mit `out result` übergeben werden.

Der entscheidende Unterschied zu `ref` ist die Verpflichtung der Methode: **Ein `out`-Parameter muss auf jedem normal endenden Weg einen Wert zugewiesen bekommen, bevor der Aufruf beendet wird.** Der bisherige Inhalt ist keine Eingabe für die Methode. Sie muss den Parameter erst beschreiben, bevor sie seinen Wert lesen kann. Diese Regeln sowie die Übergabearten sind in der C#-Referenz zu Methodenparametern beschrieben.

Ein `out`-Argument darf bereits vorher einen Wert besitzen. Nötig ist das jedoch nicht, und die Methode darf sich nicht auf diesen alten Wert verlassen. Für das einzelne Ergebnis im Beispiel wäre ein Rückgabewert meist einfacher. Der Nutzen von `out` wird deutlicher, wenn ein Aufruf mehrere Informationen liefern soll.

### Text mit `int.TryParse` in eine Zahl umwandeln

In Termin 1 wurde `Convert.ToInt32(...)` verwendet, um geeigneten Text in eine ganze Zahl umzuwandeln. Eine ungültige Eingabe kann dabei einen Laufzeitfehler auslösen. **`int.TryParse`** meldet dagegen über einen Wahrheitswert, ob die Umwandlung gelungen ist:

```
string input = "42";
bool success = int.TryParse(input, out int number);
if (success)
{
    Console.WriteLine(number + 1);
}
else
{
    Console.WriteLine("Keine gültige ganze Zahl.");
}
```

Die Methode liefert zwei Informationen auf unterschiedlichen Wegen:

- Der **Rückgabewert** wird in `success` gespeichert und zeigt den Erfolg der Umwandlung an.
- Der **`out`-Parameter** schreibt bei Erfolg die umgewandelte Zahl in `number`.

| Inhalt von `input` | `success` | `number` nach dem Aufruf | Ausgabe des Programms |
|---|---|---|---|
| `"42"` | `true` | `42` | `43` |
| `"Hallo"` | `false` | `0` | `Keine gültige ganze Zahl.` |
| `"0"` | `true` | `0` | `1` |

Gerade die letzten beiden Fälle zeigen, weshalb der Rückgabewert geprüft werden muss. `number == 0` würde nicht zwischen einer fehlgeschlagenen Umwandlung und einer gültig eingegebenen Null unterscheiden. Bei dieser Verwendung von `int.TryParse` wird `number` auch im Fehlerfall ein Wert zugewiesen, nämlich `0`; als umgewandelte Eingabe wird der Wert nur bei Erfolg verwendet.

Die Prüfung betrifft das Zahlenformat und den Wertebereich von `int`. Ob eine Zahl fachlich zulässig ist, muss gegebenenfalls zusätzlich geprüft werden. Beispielsweise kann eine negative Zahl eine gültige ganze Zahl sein, obwohl sie als Alter nicht sinnvoll wäre.

### Zwei Ergebnisse mit `out` liefern

```
GetMinMax(10, 4, out int minimum, out int maximum);
Console.WriteLine(minimum);
Console.WriteLine(maximum);

void GetMinMax(int first, int second,
    out int min, out int max)
{
    min = first;
    max = second;
    if (first > second)
    {
        min = second;
        max = first;
```

```
    }
}
```

Die Ausgabe lautet `4` und danach `10`. Zunächst erhält `min` den ersten und `max` den zweiten Eingabewert. Ist der erste Wert größer, werden die Zuordnungen korrigiert.

Beide Ausgabeparameter erhalten bereits vor der Bedingung einen Wert. Damit ist die Zuweisung auch dann sichergestellt, wenn die Bedingung falsch ist. Für die Eingaben `4` und `10` bleiben die Startzuweisungen bestehen; für `4` und `4` liefern beide Parameter denselben Wert.

Die Parameterliste ist hier nur aus Gründen der Lesbarkeit auf zwei Zeilen verteilt. Die Zeilenumbrüche verändern ihre Bedeutung nicht. Obwohl die Methode den Rückgabetyp `void` besitzt, kann sie über ihre `out`-Parameter Ergebnisse bereitstellen.

## 15. Wertübergabe, `ref` und `out` vergleichen

Die bisherigen Übergabearten unterscheiden sich darin, wie sie eine Variable des aufrufenden Codes behandeln:

| Parameter in der Definition | Wert vor dem Aufruf nötig? | Wirkung einer Zuweisung an den Parameter |
|---|---|---|
| `int value` | Ja, das Argument muss einen auswertbaren Wert liefern. | Ändert nur die lokale Kopie. |
| `ref int value` | Ja, die übergebene Variable muss bereits einen Wert besitzen. | Ändert die Variable des Aufrufers. |
| `out int value` | Nein. | Schreibt in die Variable des Aufrufers; die Methode muss vor der Rückkehr einen Wert zuweisen. |

Die Tabelle bezieht sich auf `int` und auf Zuweisungen an den jeweiligen Parameter. **`return` ist keine weitere Übergabeart für Eingaben**, sondern liefert den Ergebniswert eines Aufrufs zurück. Es kann wie bei `int.TryParse` gemeinsam mit `out` verwendet werden.

Für ein einzelnes berechnetes Ergebnis ist ein Rückgabewert meist gut lesbar: `number = Increase(number);` zeigt sowohl die Berechnung als auch die anschließende Veränderung im Hauptprogramm. `ref` eignet sich, wenn eine vorhandene Variable direkt geändert werden soll. `out` kennzeichnet eine Ergebnisvariable, deren bisherigen Wert die Methode nicht benötigt.

## 16. `in`: Eine Variable schreibgeschützt übergeben

Eine weitere Übergabeart ist **`in`**. Wie bei `ref` kann der Parameter hier auf die übergebene Variable zugreifen. Über diesen Parameter darf die Methode ihr jedoch keinen neuen Wert zuweisen:

```
int number = 5;
PrintValue(in number);
```

```
void PrintValue(in int value)
{
    Console.WriteLine(value);
}
```

Die Ausgabe lautet `5`. Eine Anweisung wie `value = 6;` innerhalb von `PrintValue` wäre ein Compilerfehler. `in` kennzeichnet eine **schreibgeschützte Referenz**. Bei einem normalen `int`-Parameter wäre eine Zuweisung an die lokale Kopie erlaubt; bei `ref int` würde sie die Variable des Aufrufers ändern. Bei `in int` ist sie nicht erlaubt.

Im gezeigten Aufruf wird `in` ausdrücklich geschrieben, damit die Übergabeart sichtbar ist. In vielen Aufrufen eines `in`-Parameters darf das Schlüsselwort an der Aufrufstelle auch entfallen.

Für einen kleinen Wert wie `int` ist durch diese Schreibweise kein praktischer Geschwindigkeitsvorteil zu erwarten. Ein Einsatzgebiet sind größere Werttypen, bei denen das Kopieren der Daten aufwendiger sein kann. Die tatsächliche Wirkung hängt vom konkreten Code ab; `in` ist kein allgemeines Mittel, um Programme schneller zu machen. Für diesen Termin steht das Erkennen des Schreibschutzes am Parameter im Vordergrund.

## 17. Rekursion: Eine Methode ruft sich selbst auf

Eine Methode kann nicht nur andere Methoden, sondern auch sich selbst aufrufen. Das heißt **Rekursion**. Das bisherige Aufrufprinzip bleibt erhalten: Ein Aufruf beginnt, führt seine Anweisungen aus und kehrt mit seinem Ergebnis zur aufrufenden Stelle zurück.

Als Beispiel soll die Summe aller ganzen Zahlen von `1` bis `n` berechnet werden. Für `n = 4` ist das `1 + 2 + 3 + 4 = 10`. Die Summe bis vier lässt sich aus der Summe bis drei bilden:

```
SumUpTo(4) = 4 + SumUpTo(3)
SumUpTo(3) = 3 + SumUpTo(2)
SumUpTo(2) = 2 + SumUpTo(1)
SumUpTo(1) = 1
```

Jeder Aufruf bearbeitet damit eine kleinere Variante derselben Aufgabe. Bei `n = 1` steht das Ergebnis direkt fest. Dieser Fall beendet die Folge weiterer Aufrufe.

### Basisfall und rekursiver Schritt

```
Console.WriteLine(SumUpTo(4));

int SumUpTo(int n)
{
    if (n == 1)
    {
        return 1;
    }
```

```
    return n + SumUpTo(n - 1);
}
```

Für dieses Beispiel gilt die **Voraussetzung** `n >= 1`. Zwei Bestandteile sind entscheidend:

- Der **Basisfall** ist `n == 1`. Die Methode gibt direkt `1` zurück und ruft sich nicht erneut auf.
- Der **rekursive Schritt** ist `n + SumUpTo(n - 1)`. Ein neuer Aufruf berechnet zunächst die kleinere Summe. Dessen Ergebnis wird danach zu `n` addiert.

Ein Basisfall muss nicht nur im Code stehen, sondern auch erreicht werden. Für positive ganze Zahlen wird `n` durch `n - 1` bei jedem weiteren Aufruf kleiner und erreicht schließlich `1`. Für null oder negative Zahlen würde diese Folge sich von `1` entfernen. Solche Eingaben erfüllen die vereinbarte Voraussetzung nicht und werden hier nicht ausgeführt.

### Die Aufrufe bauen sich auf

Beim Start von `SumUpTo(4)` steht das Ergebnis noch nicht fest. Vor der Addition wird zunächst `SumUpTo(3)` benötigt. Dieser Aufruf wartet wiederum auf `SumUpTo(2)`:

```
SumUpTo(4) wartet auf 4 + SumUpTo(3)
  SumUpTo(3) wartet auf 3 + SumUpTo(2)
    SumUpTo(2) wartet auf 2 + SumUpTo(1)
      SumUpTo(1) liefert 1
```

**Jeder Aufruf besitzt einen eigenen Parameter `n`.** Während `SumUpTo(3)` läuft, behält der wartende Aufruf `SumUpTo(4)` seinen Wert `4`. Der Ausdruck `n - 1` berechnet das Argument für den nächsten Aufruf; er weist dem bisherigen Parameter keinen neuen Wert zu.

Die noch laufenden Aufrufe und ihre Fortsetzungsstellen werden in einem **Aufrufstapel**, dem **Call Stack**, verwaltet. Als Denkmodell eignen sich übereinandergelegte Karten: Für jeden neuen Aufruf kommt eine Karte hinzu. Zunächst wird die zuletzt hinzugefügte Karte fertig bearbeitet.

### Die Ergebnisse kehren zurück

Sobald der Basisfall `1` zurückgibt, können die wartenden Additionen nacheinander abgeschlossen werden:

```
      SumUpTo(1) liefert 1
    SumUpTo(2) berechnet 2 + 1 = 3
  SumUpTo(3) berechnet 3 + 3 = 6
SumUpTo(4) berechnet 4 + 6 = 10
```

Die Einrückung zeigt dieselben Aufrufe wie beim Aufbau. Die Rückgaben führen von innen wieder nach außen. Jede fertige Addition liefert das Ergebnis für den noch wartenden Aufruf.

Erst nachdem `SumUpTo(4)` den Wert `10` zurückgegeben hat, wird `Console.WriteLine` ausgeführt und gibt `10` aus. Die rekursive Methode selbst enthält keine Ausgabe. Für `SumUpTo(1)` wird direkt der

Basisfall ausgeführt; für `SumUpTo(3)` entstehen die Rückgabewerte `1`, `3` und schließlich `6`.

### Schleife und Rekursion vergleichen

Die gleiche Summe kann auch mit einer Schleife berechnet werden. Der Vergleich knüpft an die bereits bekannten Schleifen an:

| Frage | Schleife für die Summe | Gezeigte rekursive Summe |
|---|---|---|
| Wie wird wiederholt? | Durch den nächsten Schleifendurchlauf. | Durch einen neuen Methodenaufruf. |
| Wo wird der Zwischenstand gehalten? | In einem Zähler und einer laufenden Summe. | In den eigenen Werten der noch laufenden Aufrufe. |
| Wann endet die Wiederholung? | Wenn die Schleifenbedingung `false` wird. | Wenn der Basisfall erreicht wird. |
| Wie entwickelt sich der zusätzliche Speicherbedarf? | Eine feste Anzahl Variablen genügt. | Der Aufrufstapel wächst hier mit der Aufruftiefe. |

Für eine einfache laufende Summe ist eine Schleife meist naheliegend. Rekursion ist besonders dann hilfreich, wenn eine Struktur selbst aus kleineren gleichartigen Strukturen besteht, etwa ein Ordner aus Dateien und weiteren Unterordnern. Das Summenbeispiel dient dazu, den Aufbau und die Rückkehr rekursiver Aufrufe überschaubar zu machen.

Der Aufrufstapel hat eine begrenzte Größe. Wird der Basisfall nicht erreicht oder ist die Aufruftiefe zu groß, kann ein **Stacküberlauf** entstehen. Ein solcher Fehler wird nicht absichtlich ausprobiert. Auch bei erreichbarem Basisfall können große Summen zusätzlich den Wertebereich von `int` überschreiten. Der mathematische Ansatz allein garantiert daher nicht, dass beliebig große Eingaben mit diesem Programm verarbeitet werden können.

## 18. Zusammenfassung

Die wichtigsten Erkenntnisse lassen sich wie folgt zusammenfassen:

- Eine **Definition** legt die Anweisungen eines Bausteins fest. Ein **Aufruf** führt sie aus. Die Aufrufe bestimmen die Reihenfolge im Hauptprogramm.
- Nach dem Ende einer Methode geht es an der aufrufenden Stelle weiter. Ein Aufruf innerhalb eines Ausdrucks muss zuerst sein Ergebnis liefern, bevor der umgebende Ausdruck fertig ausgewertet werden kann.
- **Parameter** stehen in der Definition, **Argumente** beim Aufruf. Anzahl, Reihenfolge und Typen müssen zusammenpassen.
- `return` beendet den aktuellen Aufruf und liefert bei einer Methode mit Rückgabewert ein Ergebnis. `void` bedeutet, dass kein Rückgabewert geliefert wird.
- **Konsolenausgabe und Rückgabe sind verschiedene Vorgänge.** Ein Rückgabewert wird nur sichtbar oder weiterverarbeitet, wenn der aufrufende Code ihn entsprechend verwendet.
- Parameter und lokale Variablen gehören zum jeweiligen Aufruf. Ihr **Gültigkeitsbereich** begrenzt, wo ihre Namen verwendet werden dürfen.

- Ein normaler `int`-Parameter erhält eine **Wertkopie**. Eine Zuweisung an ihn verändert die äußere Variable nicht. Ein zurückgegebener neuer Wert kann dort ausdrücklich zugewiesen werden.
- `ref` ermöglicht die Veränderung einer vorhandenen Variable. `out` verlangt die Zuweisung eines Ergebnisses vor der Rückkehr. `in` ermöglicht im gezeigten Beispiel einen schreibgeschützten Zugriff auf die übergebene Variable.
- Bei der **Rekursion** ruft sich eine Methode selbst auf. Ein erreichbarer **Basisfall** beendet weitere Aufrufe; danach werden die wartenden Berechnungen mit den zurückgegebenen Ergebnissen abgeschlossen.

Wer den Ablauf nachvollzieht, kann für jeden Schritt erklären, welche Werte hineingehen, welche Anweisungen ausgeführt werden und wohin das Ergebnis gelangt. Diese Verbindung zwischen Programmablauf und Datenfluss bildet die Grundlage für eigene verständliche Methoden.